

侧伴移动端软件 V1.0.0 源代码

侧伴移动端软件 V1.0.0 第 1 页

```
// ===== 文件: app/_layout.tsx =====
import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import { AuthProvider } from '@lib/auth/auth-context';
import { TouchableOpacity, Text, Platform } from 'react-native';
import { useRouter } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

function CustomBackButton() {
  const router = useRouter();
  return (
    <TouchableOpacity
      onPress={() => router.push('/(tabs)/courses')}
      style={{ marginLeft: Platform.OS === 'web' ? 10 : 0 }}
    >
      {Platform.OS === 'web' ? (
        <Text style={{ color: '#5b9bd5', fontSize: 16 }}>返回</Text>
      ) : (
        <Ionicons name="chevron-back" size={24} color="#5b9bd5" />
      )}
    </TouchableOpacity>
  );
}

export default function RootLayout() {
  return (
    <SafeAreaProvider>
      <AuthProvider>
        <StatusBar style="auto" />
        <Stack screenOptions={{ headerShown: false }}>
          <Stack.Screen name="(tabs)" />
          <Stack.Screen name="classroom/[id]" options={{ headerShown: true, title: '课程详情', headerLeft: ()
=> <CustomBackButton /> }} />
          <Stack.Screen name="classroom/create" options={{ headerShown: true, title: '创建课程', headerLeft:
() => <CustomBackButton /> }} />
          <Stack.Screen name="auth/login" />
          <Stack.Screen name="auth/register" />
          <Stack.Screen name="wallet" />
          <Stack.Screen name="enterprise" />
        </Stack>
      </AuthProvider>
    </SafeAreaProvider>
  );
}

// ===== 文件: app/(tabs)/_layout.tsx =====
import { Tabs, Redirect } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';
import { useAuth } from '@lib/auth/auth-context';
import { Colors, Spacing } from '@lib/constants/theme';
import { useFeedback } from '@lib/hooks/use-feedback';
```

侧伴移动端软件 V1.0.0 第 2 页

```
export default function TabLayout() {
  const { isAuthenticated, isLoading } = useAuth();
  const { onPress } = useFeedback();
```

```

// 加载中显示空白
if (isLoading) {
  return null;
}

// 未认证则重定向到登录页
if (!isAuthenticated) {
  return <Redirect href="/auth/login" />;
}

return (
  <Tabs
    screenOptions={{
      tabBarActiveTintColor: Colors.primary.main,
      tabBarInactiveTintColor: Colors.neutral.textSecondary,
      headerShown: true,
      tabBarStyle: {
        backgroundColor: Colors.neutral.card,
        borderTopWidth: 1,
        borderTopColor: Colors.neutral.border,
        paddingTop: Spacing.sm,
        paddingBottom: Spacing.sm,
        height: 60,
      },
      tabBarLabelStyle: {
        fontSize: 12,
        fontWeight: '500',
      },
    }}
  >
    <Tabs.Screen
      name="index"
      options={{
        title: '工作台',
        tabBarIcon: ({ color, size, focused }) => (
          <Ionicons
            name={focused ? 'grid' : 'grid-outline'}
            size={size}
            color={color}
          />
        ),
      }}
      listeners={{
        tabPress: () => onPress(),
      }}
    />

```

侧伴移动端软件 V1.0.0 第 3 页

```

<Tabs.Screen
  name="discover"
  options={{
    title: '发现',
    tabBarIcon: ({ color, size, focused }) => (
      <Ionicons
        name={focused ? 'search' : 'search-outline'}
        size={size}
        color={color}
      />
    ),
  }}
  listeners={{
    tabPress: () => onPress(),
  }}
/>
<Tabs.Screen
  name="profile"
  options={{

```

```

        title: '我的',
        tabBarIcon: ({ color, size, focused }) => (
          <Ionicons
            name={focused ? 'person' : 'person-outline'}
            size={size}
            color={color}
          />
        ),
      ]},
      listeners={{
        tabPress: () => onPress(),
      }}
    />
  <Tabs.Screen
    name="knowledge"
    options={{
      title: '知识库',
      tabBarIcon: ({ color, size, focused }) => (
        <Ionicons
          name={focused ? 'book' : 'book-outline'}
          size={size}
          color={color}
        />
      ),
    }}
    listeners={{
      tabPress: () => onPress(),
    }}
  />
  { /* 隐藏其他tab页面，通过工作台入口访问 */ }
  <Tabs.Screen

```

侧伴移动端软件 V1.0.0 第 4 页

```

    name="courses"
    options={{
      title: '我的课程',
      headerShown: true,
      href: null,
    }}
  />
  <Tabs.Screen
    name="questions"
    options={{ title: '问答', headerShown: true, href: null }}
  />
  <Tabs.Screen
    name="notes"
    options={{ title: '笔记', headerShown: true, href: null }}
  />
  <Tabs.Screen
    name="buddy"
    options={{ title: '搭子', headerShown: true, href: null }}
  />
  <Tabs.Screen
    name="matching"
    options={{ title: '匹配', headerShown: true, href: null }}
  />
  <Tabs.Screen
    name="gamification"
    options={{ title: '成长', headerShown: true, href: null }}
  />
  <Tabs.Screen
    name="invite"
    options={{ title: '邀请', headerShown: true, href: null }}
  />
  <Tabs.Screen
    name="payment"
    options={{ title: '充值', headerShown: true, href: null }}
  />

```

```

    />
  </Tabs>
);
}
// ===== 文件: app/(tabs)/index.tsx =====
import { View, Text, TouchableOpacity, StyleSheet, ScrollView, Animated } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '@lib/auth/auth-context';
import { Ionicons } from '@expo/vector-icons';
import { Rounded, Spacing } from '@lib/constants/theme';
import { useFeedback } from '@lib/hooks/use-feedback';
import { useHaptics } from '@lib/hooks/use-haptics';
import { useRef, useState } from 'react';
import { useI18n } from '@lib/i18n';

// iOS 风格颜色系统

```

侧伴移动端软件 V1.0.0 第 5 页

```

const iOSColors = {
  bg: 'transparent', // 使用渐变背景
  bgSolid: '#f5f3f2',
  surface: 'rgba(255, 255, 255, 0.55)',
  surfaceSolid: 'FFFFFF',
  fg: '#1a1a1a',
  muted: '#666666',
  border: 'rgba(230, 225, 220, 0.6)',
  accent: '#c45a1a',
  accentLight: '#fde8e0',
  secondary: '#1a8a8a',
  secondaryLight: '#e8f5f5',
};

// 快捷功能配置 (10个)
const quickFunctions = [
  { key: 'courses', titleKey: 'home.myCourses', icon: 'book', color: '#f45a1a', bgColor: '#fce8e0', route: '/courses' },
  { key: 'qa', titleKey: 'home.qaBounty', icon: 'help-circle', color: '#d97706', bgColor: '#fef3c7', route: '/questions' },
  { key: 'notes', titleKey: 'home.sharedNotes', icon: 'document-text', color: '#14b8a6', bgColor: '#e8f5f5', route: '/notes' },
  { key: 'buddy', titleKey: 'home.studyBuddy', icon: 'happy', color: '#2563eb', bgColor: '#dbeafe', route: '/buddy' },
  { key: 'matching', titleKey: 'home.studyMatching', icon: 'people', color: '#8b5cf6', bgColor: '#ede9fe', route: '/matching' },
  { key: 'growth', titleKey: 'home.growthSystem', icon: 'trending-up', color: '#f45a1a', bgColor: '#fce8e0', route: '/gamification' },
  { key: 'invite', titleKey: 'home.inviteRewards', icon: 'gift', color: '#d97706', bgColor: '#fef3c7', route: '/invite' },
  { key: 'recharge', titleKey: 'home.recharge', icon: 'card', color: '#14b8a6', bgColor: '#e8f5f5', route: '/payment' },
  { key: 'wallet', titleKey: 'home.wallet', icon: 'cash', color: '#2563eb', bgColor: '#dbeafe', route: '/wallet' },
  { key: 'enterprise', titleKey: 'home.enterpriseServices', icon: 'briefcase', color: '#8b5cf6', bgColor: '#ede9fe', route: '/enterprise' },
];

// 近期课程数据
const recentCourses = [
  { id: 1, name: '数据分析基础', section: '第 12 节 · 数据可视化', progress: 68, icon: '🇮🇹', color: '#fce8e0' },
  { id: 2, name: 'Python 编程入门', section: '第 8 节 · 函数与模块', progress: 45, icon: '🇩🇪', color: '#dbeafe' },
  { id: 3, name: 'UI 设计原理', section: '第 5 节 · 色彩与排版', progress: 28, icon: '🇧🇪', color: '#ede9fe' },
];

// 推荐课程数据
const recommendedCourses = [
  { id: 1, name: '机器学习概论', category: '人工智能', icon: '🇨🇳' },

```

```

    { id: 2, name: '商业数据分析', category: '商业分析', icon: '📊' },
    { id: 3, name: '写作与表达', category: '人文素养', icon: '📝' },
    { id: 4, name: '科学思维方法', category: '思维方式', icon: '🧠' },
  ];

// 笔记分类数据
const notesCategories = [
  { key: 'all', titleKey: 'home.allNotes', icon: '📁', count: '32 条' },
  { key: 'fav', titleKey: 'home.favorites', icon: '⭐', count: '12 条' },
  { key: 'today', titleKey: 'home.today', icon: '📅', count: '3 条' },
  { key: 'new', titleKey: 'home.quickRecord', icon: '+', count: '' },
];

```

侧伴移动端软件 V1.0.0 第 6 页

```

// 快捷功能按钮组件
function QuickFunctionBtn({ item, onPress, t }: { item: typeof quickFunctions[0]; onPress: () => void; t:
(key: string) => string }) {
  const scaleAnim = useRef(new Animated.Value(1)).current;

  const handlePressIn = () => {
    Animated.spring(scaleAnim, {
      toValue: 0.92,
      useNativeDriver: true,
    }).start();
  };

  const handlePressOut = () => {
    Animated.spring(scaleAnim, {
      toValue: 1,
      useNativeDriver: true,
    }).start();
  };

  return (
    <TouchableOpacity
      onPress={onPress}
      onPressIn={handlePressIn}
      onPressOut={handlePressOut}
      activeOpacity={0.9}
      style={styles.quickFnBtn}
    >
      <Animated.View style={{ transform: [{ scale: scaleAnim }], alignItems: 'center' }}>
        <View style={[styles.quickFnIcon, { backgroundColor: item.bgColor }]}>
          <Icons name={item.icon as any} size={16} color={item.color} />
        </View>
        <Text style={styles.quickFnLabel}>{t(item.titleKey)}</Text>
      </Animated.View>
    </TouchableOpacity>
  );
}

// 课程项组件（近期课程列表）
function CourseItem({ course }: { course: typeof recentCourses[0] }) {
  const router = useRouter();

  return (
    <TouchableOpacity
      style={styles.courseItem}
      onPress={() => router.push(`/course/${course.id}` as any)}
      activeOpacity={0.85}
    >
      <View style={[styles.courseIcon, { backgroundColor: course.color }]}>
        <Text style={styles.courseIconText}>{course.icon}</Text>
      </View>
    </TouchableOpacity>
  );
}

```

```

    <View style={styles.courseInfo}>
      <Text style={styles.courseName}>{course.name}</Text>
      <Text style={styles.courseMeta}>{course.section}</Text>
      <View style={styles.courseProgress}>
        <View style={styles.progressBar}>
          <View style={[styles.progressFill, { width: `${course.progress}%` }]} />
        </View>
        <Text style={styles.progressNum}>{course.progress}%</Text>
      </View>
    </View>
  </TouchableOpacity>
);
}

// 推荐课程卡片组件
function RecommendedCard({ course }: { course: typeof recommendedCourses[0] }) {
  const router = useRouter();
  const scaleAnim = useRef(new Animated.Value(1)).current;

  const handlePressIn = () => {
    Animated.spring(scaleAnim, {
      toValue: 0.95,
      useNativeDriver: true,
    }).start();
  };

  const handlePressOut = () => {
    Animated.spring(scaleAnim, {
      toValue: 1,
      useNativeDriver: true,
    }).start();
  };

  return (
    <TouchableOpacity
      onPress={() => router.push(`/course/${course.id}` as any)}
      onPressIn={handlePressIn}
      onPressOut={handlePressOut}
      activeOpacity={0.9}
    >
      <Animated.View style={[styles.recommendedCard, { transform: [{ scale: scaleAnim }] }]}>
        <View style={[styles.recIcon, { backgroundColor: iosColors.accentLight }]}>
          <Text style={styles.recIconText}>{course.icon}</Text>
        </View>
        <Text style={styles.recName}>{course.name}</Text>
        <Text style={styles.recCat}>{course.category}</Text>
      </Animated.View>
    </TouchableOpacity>
  );
}

```

```

// 笔记卡片组件
function NoteCard({ note, t }: { note: typeof notesCategories[0]; t: (key: string) => string }) {
  const router = useRouter();
  const scaleAnim = useRef(new Animated.Value(1)).current;

  const handlePressIn = () => {
    Animated.spring(scaleAnim, {
      toValue: 0.95,
      useNativeDriver: true,
    }).start();
  };

```

```

const handlePressOut = () => {
  Animated.spring(scaleAnim, {
    toValue: 1,
    useNativeDriver: true,
  }).start();
};

return (
  <TouchableOpacity
    style={styles.noteCard}
    onPress={() => {
      if (note.key === 'new') {
        router.push('/notes/new' as any);
      } else {
        router.push('/notes' as any);
      }
    }}
    onPressIn={handlePressIn}
    onPressOut={handlePressOut}
    activeOpacity={0.9}
  >
    <Animated.View style={{ transform: [{ scale: scaleAnim }], alignItems: 'center' }}>
      <Text style={styles.noteIcon}>{note.icon}</Text>
      <Text style={styles.noteLabel}>{t(note.titleKey)}</Text>
      {note.count && <Text style={styles.noteCount}>{note.count}</Text>}
    </Animated.View>
  </TouchableOpacity>
);
}

export default function HomeScreen() {
  const router = useRouter();
  const { user } = useAuth();
  const { onPress } = useFeedback();
  const haptics = useHaptics();
  const { t } = useI18n();
  const [notifExpanded, setNotifExpanded] = useState(false);

```

侧伴移动端软件 V1.0.0 第 9 页

```

const handlePress = (route: string) => {
  haptics.light();
  onPress();
  router.push(route as any);
};

return (
  <View style={styles.container}>
    <ScrollView style={styles.scrollView} showsVerticalScrollIndicator={false}>
      /* Header 用户区 */
      <View style={styles.header}>
        <View style={styles.avatar}>
          <Text style={styles.avatarText}>{(user?.nickname || user?.email || '用户').charAt(0)}</Text>
        </View>
        <View style={styles.userInfo}>
          <Text style={styles.userName}>{user?.nickname || user?.email?.split('@')[0] || '林小雨'}</Text>
          <Text style={styles.userStats}>
            <Text style={styles.statValue}>23</Text>{t('home.days')} .
            <Text style={styles.statValue}>8</Text>{t('home.courses')} .
            <Text style={styles.statValue}>156</Text>{t('home.hours')}
          </Text>
        </View>
        <TouchableOpacity style={styles.headerBtn} onPress={() => router.push('/profile')}>
          <Ionicons name="settings-outline" size={18} color={iOSColors.fg} />
        </TouchableOpacity>
      </View>

```

```

{/* Streak 连续学习卡片 */}
<View style={styles.streakCard}>
  <View>
    <Text style={styles.streakNumber}>23</Text>
    <Text style={styles.streakText}>{t('home.streakDays')}</Text>
  </View>
  <View style={styles.streakDots}>
    {[1,2,3,4,5,6,7].map(i => (
      <View key={i} style={[[styles.dot, i <= 5 && styles.dotActive]]} />
    ))}
  </View>
</View>

{/* Notification 通知卡片 */}
<TouchableOpacity
  style={[styles.notificationCard, notifExpanded && styles.notificationCardExpanded]}
  onPress={() => setNotifExpanded(!notifExpanded)}
  activeOpacity={0.9}
>
  <View style={styles.notifHeader}>
    <View style={styles.notifIcon}>
      <Text style={styles.notifIconEmoji}>🔔</Text>
    </View>
  </View>

```

侧伴移动端软件 V1.0.0 第 10 页

```

    <View style={styles.notifBadge}>
      <Text style={styles.notifBadgeText}>3</Text>
    </View>
  </View>
  <View style={styles.notifSummary}>
    <Text style={styles.notifTitle}>{t('home.newLessonReminder')} · 数据分析基础</Text>
    <Text style={styles.notifTime}>5分钟前</Text>
  </View>
  <View style={styles.notifChevron}>
    <Ionicons
      name={notifExpanded ? 'chevron-up' : 'chevron-down'}
      size={14}
      color={iOSColors.muted}
    />
  </View>
</View>
{notifExpanded && (
  <View style={styles.notifDetail}>
    <Text style={styles.notifBody}>
      第 13 节「交互式仪表盘」已更新，包含 2 个实战案例和 1 个随堂测验。完成本节后将解锁下一章「高级筛选」。
    </Text>
    <View style={styles.notifActions}>
      <TouchableOpacity style={styles.notifActionPrimary} onPress={() => router.push('/courses')}>
        <Text style={styles.notifActionPrimaryText}>{t('home.startLearning')}</Text>
      </TouchableOpacity>
      <TouchableOpacity style={styles.notifActionSecondary}>
        <Text style={styles.notifActionSecondaryText}>{t('home.remindLater')}</Text>
      </TouchableOpacity>
    </View>
  </View>
)}
</TouchableOpacity>

{/* Quick Functions 快捷功能 */}
<View style={styles.quickFunctions}>
  <View style={styles.quickFunctionsGrid}>
    {quickFunctions.map((item) => (
      <QuickFunctionBtn
        key={item.key}
        item={item}
        onPress={() => handlePress(item.route)}
        t={t}
      />
    ))}
  </View>

```



```

    />
  )}}
</View>
</View>

{/* Recent Courses 近期课程 */}
<View style={styles.section}>
  <View style={styles.sectionHeader}>

```

侧伴移动端软件 V1.0.0 第 11 页

```

    <Text style={styles.sectionTitle}>{t('home.recentCourses')}</Text>
    <TouchableOpacity onPress={() => router.push('/courses')}>
      <Text style={styles.sectionLink}>全部</Text>
    </TouchableOpacity>
  </View>
  <View style={styles.courseList}>
    {recentCourses.map(course => (
      <CourseItem key={course.id} course={course} />
    ))}
  </View>
</View>

{/* Recommended 推荐课程 */}
<View style={styles.section}>
  <View style={styles.sectionHeader}>
    <Text style={styles.sectionTitle}>{t('home.recommendedCourses')}</Text>
    <TouchableOpacity onPress={() => router.push('/courses')}>
      <Text style={styles.sectionLink}>更多</Text>
    </TouchableOpacity>
  </View>
  <ScrollView
    horizontal
    showsHorizontalScrollIndicator={false}
    style={styles.recommendedScroll}
  >
    {recommendedCourses.map(course => (
      <RecommendedCard key={course.id} course={course} />
    ))}
  </ScrollView>
</View>

{/* Notes 我的笔记 */}
<View style={styles.section}>
  <View style={styles.sectionHeader}>
    <Text style={styles.sectionTitle}>我的笔记</Text>
    <TouchableOpacity onPress={() => router.push('/notes')}>
      <Text style={styles.sectionLink}>全部</Text>
    </TouchableOpacity>
  </View>
  <View style={styles.notesGrid}>
    {notesCategories.map(note => (
      <NoteCard key={note.key} note={note} t={t} />
    ))}
  </View>
</View>
</ScrollView>
</View>
);
}

```

侧伴移动端软件 V1.0.0 第 12 页

```
const styles = StyleSheet.create({
```

```

container: {
  flex: 1,
  backgroundColor: iOSColors.bgSolid,
},
scrollView: {
  flex: 1,
  paddingHorizontal: Spacing.md,
},
// Header 用户区
header: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: Spacing.sm,
  paddingTop: Spacing.md,
  paddingBottom: Spacing.sm,
  marginBottom: Spacing.lg,
},
avatar: {
  width: 44,
  height: 44,
  borderRadius: 22,
  backgroundColor: iOSColors.accent,
  alignItems: 'center',
  justifyContent: 'center',
},
avatarText: {
  color: '#fff',
  fontWeight: '600',
  fontSize: 18,
},
userInfo: {
  flex: 1,
},
userName: {
  fontSize: 18,
  fontWeight: '600',
  color: iOSColors.fg,
  letterSpacing: -0.3,
},
userStats: {
  fontSize: 12,
  color: iOSColors.muted,
  marginTop: 2,
},
statValue: {
  color: iOSColors.accent,
  fontWeight: '600',
},
headerBtn: {

```

侧伴移动端软件 V1.0.0 第 13 页

```

  width: 44,
  height: 44,
  borderRadius: 22,
  backgroundColor: 'rgba(255, 255, 255, 0.5)',
  alignItems: 'center',
  justifyContent: 'center',
  borderWidth: 0.5,
  borderColor: iOSColors.border,
},
// Streak 连续学习
streakCard: {
  backgroundColor: '#c45a1a',
  borderRadius: Rounded.lg,
  padding: Spacing.lg,
  marginBottom: Spacing.lg,
  flexDirection: 'row',

```

```

        alignItems: 'center',
        justifyContent: 'space-between',
    },
    streakNumber: {
        fontSize: 32,
        fontWeight: '700',
        color: '#fff',
        letterSpacing: -1,
    },
    streakText: {
        fontSize: 11,
        color: '#fff',
        opacity: 0.85,
        marginTop: 2,
    },
    streakDots: {
        flexDirection: 'row',
        gap: 5,
    },
    dot: {
        width: 5,
        height: 5,
        borderRadius: 2.5,
        backgroundColor: 'rgba(255,255,255,0.25)',
    },
    dotActive: {
        backgroundColor: '#fff',
    },
    // Notification 通知卡片
    notificationCard: {
        backgroundColor: iOSColors.surface,
        borderRadius: Rounded.md,
        borderWidth: 0.5,
        borderColor: iOSColors.border,
    },

```

侧伴移动端软件 V1.0.0 第 14 页

```

        marginBottom: Spacing.lg,
        overflow: 'hidden',
    },
    notificationCardExpanded: {
        backgroundColor: iOSColors.surfaceSolid,
    },
    notifHeader: {
        flexDirection: 'row',
        alignItems: 'center',
        gap: Spacing.sm,
        padding: Spacing.sm,
        minHeight: 44,
    },
    notifIcon: {
        width: 36,
        height: 36,
        borderRadius: 10,
        backgroundColor: iOSColors.accentLight,
        alignItems: 'center',
        justifyContent: 'center',
        position: 'relative',
    },
    notifIconEmoji: {
        fontSize: 16,
    },
    notifBadge: {
        position: 'absolute',
        top: -3,
        right: -3,
        width: 16,
        height: 16,
    },

```

```
borderRadius: 8,
backgroundColor: iOSColors.accent,
alignItems: 'center',
justifyContent: 'center',
},
notifBadgeText: {
  color: '#fff',
  fontSize: 9,
  fontWeight: '700',
},
notifSummary: {
  flex: 1,
},
notifTitle: {
  fontSize: 13,
  fontWeight: '600',
  color: iOSColors.fg,
  marginBottom: 1,
},
```

侧伴移动端软件 V1.0.0 第 15 页

```
notifTime: {
  fontSize: 10,
  color: iOSColors.muted,
  opacity: 0.7,
},
notifChevron: {
  width: 20,
  height: 20,
  alignItems: 'center',
  justifyContent: 'center',
},
notifDetail: {
  padding: Spacing.sm,
  paddingLeft: Spacing.md + 36 + Spacing.sm,
},
notifBody: {
  fontSize: 12,
  color: iOSColors.muted,
  lineHeight: 18,
  marginBottom: Spacing.xs,
},
notifActions: {
  flexDirection: 'row',
  gap: Spacing.xs,
},
notifActionPrimary: {
  backgroundColor: iOSColors.accent,
  borderRadius: 6,
  paddingHorizontal: 12,
  paddingVertical: 4,
  minHeight: 28,
  alignItems: 'center',
  justifyContent: 'center',
},
notifActionPrimaryText: {
  color: '#fff',
  fontSize: 11,
  fontWeight: '600',
},
notifActionSecondary: {
  backgroundColor: iOSColors.border,
  borderRadius: 6,
  paddingHorizontal: 12,
  paddingVertical: 4,
  minHeight: 28,
  alignItems: 'center',
```

侧伴移动端软件 V1.0.0 第 16 页

侧伴移动端软件 V1.0.0 第 17 页

```
    color: iOSColors.fg,
},
sectionLink: {
    fontSize: 12,
    color: iOSColors.accent,
```

```

        fontWeight: '500',
    },
    // Courses 课程列表
    courseList: {
        gap: Spacing.sm,
    },
    courseItem: {
        backgroundColor: iOSColors.surface,
        borderRadius: Rounded.md,
        borderWidth: 0.5,
        borderColor: iOSColors.border,
        padding: Spacing.sm,
        flexDirection: 'row',
        gap: Spacing.sm,
    },
    courseIcon: {
        width: 44,
        height: 44,
        borderRadius: Rounded.sm,
        alignItems: 'center',
        justifyContent: 'center',
    },
    courseIconText: {
        fontSize: 18,
    },
    courseInfo: {
        flex: 1,
    },
    courseName: {
        fontSize: 14,
        fontWeight: '600',
        color: iOSColors.fg,
        marginBottom: 3,
    },
    courseMeta: {
        fontSize: 11,
        color: iOSColors.muted,
    },
    courseProgress: {
        flexDirection: 'row',
        alignItems: 'center',
        gap: Spacing.xs,
        marginTop: 6,
    },
    progressBar: {

```

```

        flex: 1,
        maxWidth: 64,
        height: 3,
        backgroundColor: iOSColors.border,
        borderRadius: 2,
        overflow: 'hidden',
    },
    progressFill: {
        height: 3,
        backgroundColor: iOSColors.accent,
        borderRadius: 2,
    },
    progressNum: {
        fontSize: 11,
        color: iOSColors.muted,
        minWidth: 32,
        textAlign: 'right',
    },
    // Recommended 推荐课程
    recommendedScroll: {

```

```

        marginBottom: Spacing.lg,
      },
      recommendedCard: {
        width: 140,
        backgroundColor: iOSColors.surface,
        borderRadius: Rounded.md,
        borderWidth: 0.5,
        borderColor: iOSColors.border,
        padding: Spacing.sm,
        marginRight: Spacing.sm,
      },
      recIcon: {
        width: 36,
        height: 36,
        borderRadius: Rounded.sm,
        alignItems: 'center',
        justifyContent: 'center',
        marginBottom: Spacing.xs,
      },
      recIconText: {
        fontSize: 15,
      },
      recName: {
        fontSize: 13,
        fontWeight: '600',
        color: iOSColors.fg,
        marginBottom: 2,
      },
      recCat: {
        fontSize: 11,

```

侧伴移动端软件 V1.0.0 第 19 页

```

        color: iOSColors.muted,
      },
      // Notes 笔记网格
      notesGrid: {
        flexDirection: 'row',
        flexWrap: 'wrap',
        gap: Spacing.sm,
        marginBottom: Spacing.lg,
      },
      noteCard: {
        width: '48%',
        backgroundColor: iOSColors.surface,
        borderRadius: Rounded.md,
        borderWidth: 0.5,
        borderColor: iOSColors.border,
        padding: Spacing.md,
        alignItems: 'center',
        minHeight: 80,
      },
      noteIcon: {
        fontSize: 18,
        marginBottom: 4,
      },
      noteLabel: {
        fontSize: 13,
        fontWeight: '600',
        color: iOSColors.fg,
      },
      noteCount: {
        fontSize: 11,
        color: iOSColors.muted,
        marginTop: 1,
      },
    },
  });
// ===== 文件: app/(tabs)/buddy.tsx =====

```

```
import { View, Text, StyleSheet, FlatList, RefreshControl, TouchableOpacity, Alert, ScrollView } from
'react-native';
import { useState, useCallback, useEffect } from 'react';
import { apiClient } from '@lib/api-client';
import { Colors, Rounded, Spacing } from '@lib/constants/theme';
import { useFeedback } from '@lib/hooks/use-feedback';

interface BuddyType {
  id: string;
  name: string;
  description: string;
  tone: string;
}

interface BuddyMessage {
  id: string;
```

侧伴移动端软件 V1.0.0 第 20 页

```
trigger_event: string;
message_type: string;
content: string;
read: boolean;
created_at: string;
}

export default function BuddyScreen() {
  const { onSuccess, onError } = useFeedback();
  const [buddyTypes, setBuddyTypes] = useState<BuddyType[]>([]);
  const [myConfig, setMyConfig] = useState<any>(null);
  const [messages, setMessages] = useState<BuddyMessage[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    loadData();
  }, []);

  async function loadData() {
    setIsLoading(true);
    try {
      const [typesData, configData, messagesData] = await Promise.all([
        apiClient.getBuddyTypes(),
        apiClient.getMyBuddyConfig(),
        apiClient.getBuddyMessages(1, 20),
      ]);
      setBuddyTypes(typesData.types || []);
      setMyConfig(configData);
      setMessages(messagesData.items || []);
    } catch (error) {
      console.error('Load buddy data error:', error);
    } finally {
      setIsLoading(false);
    }
  }

  const onRefresh = useCallback(() => {
    loadData();
  }, []);

  const selectBuddyType = async (buddyType: BuddyType) => {
    try {
      await apiClient.setBuddyConfig(buddyType.id);
      onSuccess();
      Alert.alert('成功', `已选择「${buddyType.name}」作为你的学习搭子`);
      loadData();
    } catch (error) {
      onError();
      Alert.alert('失败', '设置失败, 请稍后重试');
    }
  }
}
```



```
}
```

侧伴移动端软件 V1.0.0 第 21 页

```
};

const renderMessage = ({ item }: { item: BuddyMessage }) => (
  <View style={styles.messageItem, !item.read && styles.unreadMessage}>
    <Text style={styles.messageContent}>{item.content}</Text>
    <Text style={styles.messageTime}>
      {new Date(item.created_at).toLocaleString()}
    </Text>
  </View>
);

return (
  <ScrollView
    style={styles.container}
    refreshControl={<RefreshControl refreshing={isLoading} onRefresh={onRefresh} />}
  >
    { /* 当前搭子 */ }
    <View style={styles.section}>
      <Text style={styles.sectionTitle}>我的学习搭子</Text>
      {myConfig && (
        <View style={styles.currentBuddy}>
          <Text style={styles.buddyName}>
            {myConfig.buddy_name || '鼓励者'}
          </Text>
          <Text style={styles.buddyType}>
            类型: {myConfig.buddy_type}
          </Text>
          <Text style={styles.buddyTone}>
            风格: {myConfig.tone_style}
          </Text>
        </View>
      )}
    </View>

    { /* 选择搭子类型 */ }
    <View style={styles.section}>
      <Text style={styles.sectionTitle}>选择搭子类型</Text>
      {buddyTypes.map((type) => (
        <TouchableOpacity
          key={type.id}
          style={[
            styles.typeCard,
            myConfig?.buddy_type === type.id && styles.activeTypeCard,
          ]}
          onPress={() => selectBuddyType(type)}
        >
          <Text style={styles.typeName}>{type.name}</Text>
          <Text style={styles.typeDesc}>{type.description}</Text>
        </TouchableOpacity>
      ))}
    </View>
  )}
```

侧伴移动端软件 V1.0.0 第 22 页

```
</View>

{ /* 搭子消息 */ }
<View style={styles.section}>
  <Text style={styles.sectionTitle}>搭子消息</Text>
  <FlatList
    data={messages}
    renderItem={renderMessage}
  />
</View>
```

```

        keyExtractor={item => item.id}
        scrollEnabled={false}
        ListEmptyComponent={
          <Text style={styles.emptyText}>暂无消息</Text>
        }
      />
    </View>
  </ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: Colors.neutral.background },
  section: {
    backgroundColor: Colors.neutral.card,
    marginBottom: Spacing.sm,
    padding: Spacing.md,
    borderRadius: Rounded.lg,
    marginHorizontal: Spacing.sm,
    borderWidth: 1,
    borderColor: Colors.neutral.border,
  },
  sectionTitle: { fontSize: 18, fontWeight: '600', marginBottom: Spacing.sm, color:
Colors.neutral.textPrimary },
  currentBuddy: {
    backgroundColor: Colors.neutral.backgroundAlt,
    padding: Spacing.md,
    borderRadius: Rounded.md,
    borderWidth: 1,
    borderColor: Colors.neutral.border,
  },
  buddyName: { fontSize: 20, fontWeight: 'bold', color: Colors.primary.main },
  buddyType: { fontSize: 14, color: Colors.neutral.textSecondary, marginTop: Spacing.sm - 2 },
  buddyTone: { fontSize: 14, color: Colors.neutral.textMuted },
  typeCard: {
    padding: Spacing.md,
    borderRadius: Rounded.md,
    borderWidth: 1,
    borderColor: Colors.neutral.border,
    marginBottom: Spacing.sm,
    backgroundColor: Colors.neutral.backgroundAlt,
  },
  activeTypeCard: {

```

侧伴移动端软件 V1.0.0 第 23 页

```

    borderColor: Colors.primary.main,
    backgroundColor: Colors.primary.transparent,
  },
  typeName: { fontSize: 16, fontWeight: '600', color: Colors.neutral.textPrimary },
  typeDesc: { fontSize: 14, color: Colors.neutral.textSecondary, marginTop: Spacing.sm - 2 },
  messageItem: {
    padding: Spacing.sm + 6,
    borderRadius: Rounded.md,
    backgroundColor: Colors.neutral.backgroundAlt,
    marginBottom: Spacing.sm,
    borderWidth: 1,
    borderColor: Colors.neutral.border,
  },
  unreadMessage: { backgroundColor: Colors.primary.transparent, borderColor: Colors.primary.main },
  messageContent: { fontSize: 14, color: Colors.neutral.textPrimary },
  messageTime: { fontSize: 12, color: Colors.neutral.textMuted, marginTop: Spacing.sm - 2 },
  emptyText: { color: Colors.neutral.textMuted, textAlign: 'center', padding: Spacing.lg + 4 },
});

// ===== 文件: app/(tabs)/courses.tsx =====
import { useState, useEffect, useMemo } from 'react';
import {

```

```
View,
Text,
FlatList,
TouchableOpacity,
StyleSheet,
ActivityIndicator,
Animated,
} from 'react-native';
import { useRouter } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';
import { apiClient } from '@lib/api-client';
import { Rounded, Spacing } from '@lib/constants/theme';
import { useFeedback } from '@lib/hooks/use-feedback';
import { useHaptics } from '@lib/hooks/use-haptics';
import { useRef } from 'react';
```

```
// iOS 风格颜色系统
const iOSSColors = {
  bgSolid: '#f5f3f2',
  surface: 'rgba(255, 255, 255, 0.55)',
  surfaceSolid: '#FFFFFF',
  fg: '#1a1a1a',
  muted: '#666666',
  border: 'rgba(230, 225, 220, 0.6)',
  accent: '#c45a1a',
  accentLight: '#fde8e0',
  secondary: '#1a8a8a',
  secondaryLight: '#e8f5f5',
```

侧伴移动端软件 V1.0.0 第 24 页

```
};

interface Classroom {
  id: string;
  name: string;
  description?: string;
  language_directive?: string;
  tags?: string[]; // 课程标签
  created_at: string;
  updated_at?: string;
}

// 模拟课程进度和状态数据（后续可接入真实数据）
const mockCourseData = {
  progress: Math.floor(Math.random() * 100),
  status: ['in-progress', 'completed', 'not-started'][Math.floor(Math.random() * 3)],
  category: ['数据科学', '编程基础', '设计基础', '人工智能', '商业分析', '人文素养', '思维方式', '语言学习', '数学基础', '产品设计', '前端框架'][Math.floor(Math.random() * 11)],
  totalSections: Math.floor(Math.random() * 30) + 10,
  completedSections: Math.floor(Math.random() * 20),
};

// 状态标签配置
const statusConfig = {
  'in-progress': { label: '学习中', bgColor: iOSSColors.accentLight, textColor: iOSSColors.accent },
  'completed': { label: '已完成', bgColor: iOSSColors.secondaryLight, textColor: iOSSColors.secondary },
  'not-started': { label: '未开始', bgColor: 'rgba(230, 225, 220, 0.4)', textColor: iOSSColors.muted },
};

export default function CoursesScreen() {
  const router = useRouter();
  const { onPress } = useFeedback();
  const haptics = useHaptics();
  const [classrooms, setClassrooms] = useState<Classroom[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
```

```
// 筛选状态
const [activeFilter, setActiveFilter] = useState('all');

useEffect(() => {
  loadClassrooms();
}, []);

// 筛选课程
const filteredClassrooms = useMemo(() => {
  if (activeFilter === 'all') return classrooms;

  // 根据模拟状态筛选（后续替换为真实数据）
  return classrooms.filter(() => {
    const status = mockCourseData.status;
```

侧伴移动端软件 V1.0.0 第 25 页

```
    if (activeFilter === 'progress') return status === 'in-progress';
    if (activeFilter === 'completed') return status === 'completed';
    if (activeFilter === 'notstarted') return status === 'not-started';
    return true;
  });
}, [classrooms, activeFilter]);

// 统计各状态数量（模拟数据）
const courseStats = useMemo(() => {
  const total = classrooms.length;
  const inProgress = Math.floor(total * 0.4);
  const completed = Math.floor(total * 0.35);
  const notStarted = total - inProgress - completed;
  return { total, inProgress, completed, notStarted };
}, [classrooms]);

const loadClassrooms = async () => {
  setLoading(true);
  setError(null);
  try {
    const data = await apiClient.getClassrooms();
    setClassrooms(data);
  } catch (err: any) {
    setError(err.message || '加载失败');
  } finally {
    setLoading(false);
  }
};

// 格式化日期
const formatDate = (dateStr: string) => {
  const date = new Date(dateStr);
  return date.toLocaleDateString('zh-CN', { year: 'numeric', month: '2-digit', day: '2-digit'
}).replace(/\//g, '.');
};

// 获取缩略图颜色（根据索引循环）
const getThumbColor = (index: number) => {
  const colors = ['coral', 'mint', 'gold', 'blue', 'purple'];
  return colors[index % colors.length];
};

// 课程进度缓存（内存中）
const courseProgressCache: Record<string, { progress: number; status: string }> = {};

// 获取或生成课程进度（后续接入真实数据）
function getCourseProgress(classroom: Classroom) {
  if (!courseProgressCache[classroom.id]) {
    // 模拟进度数据（后续替换为从后端获取）
    courseProgressCache[classroom.id] = {
      progress: Math.floor(Math.random() * 100),
```

```

        status: ['in-progress', 'completed', 'not-started'][Math.floor(Math.random() * 3)],
    };
}
return courseProgressCache[classroom.id];
}

// 课程卡片组件
function CourseCard({ classroom, index }: { classroom: Classroom; index: number }) {
    const scaleAnim = useRef(new Animated.Value(1)).current;
    const thumbColor = getThumbColor(index);

    // 获取课程进度（模拟数据，后续接入真实数据）
    const progressData = getCourseProgress(classroom);
    const progress = progressData.progress;
    const status = progressData.status;

    // 模拟分类和章节数（后续接入真实数据）
    const category = ['数据科学', '编程基础', '设计基础', '人工智能', '商业分析', '人文素养']
[Math.floor(Math.random() * 6)];
    const totalSections = Math.floor(Math.random() * 30) + 10;
    const completedSections = Math.floor(totalSections * progress / 100);

    const statusInfo = statusConfig[status as keyof typeof statusConfig];

    const handlePressIn = () => {
        Animated.spring(scaleAnim, {
            toValue: 0.98,
            useNativeDriver: true,
        }).start();
    };

    const handlePressOut = () => {
        Animated.spring(scaleAnim, {
            toValue: 1,
            useNativeDriver: true,
        }).start();
    };

    return (
        <TouchableOpacity
            onPress={() => router.push(`/course/${classroom.id}` as any)}
            onPressIn={handlePressIn}
            onPressOut={handlePressOut}
            activeOpacity={0.9}
        >
            <Animated.View style={[styles.courseCard, { transform: [{ scale: scaleAnim }] }]}>
                {/* 缩略图 */}
                <View style={[styles.courseThumb, { backgroundColor: thumbColor === 'coral' ? '#f45a1a' :
thumbColor === 'mint' ? '#14b8a6' : thumbColor === 'gold' ? '#f59e0b' : thumbColor === 'blue' ? '#2563eb' :
'#8b5cf6' }]}>
                    <Text style={styles.courseThumbIcon}>
                        {thumbColor === 'coral' ? '🇺🇸' : thumbColor === 'mint' ? '🇯🇵' : thumbColor === 'gold' ? '📈'
: thumbColor === 'blue' ? '📊' : '🎨'}
                    </Text>
                </View>
            </Animated.View>
        </TouchableOpacity>
    );
}

```

```

    {/* Overlay */}
    <View style={styles.courseThumbOverlay}>
        {status === 'completed' ? (
            <Text style={styles.courseThumbOverlayText}>✓</Text>
        ) : (

```

```

        <Text style={styles.courseThumbOverlayText}>{completedSections}/{totalSections}</Text>
      )}
    </View>
  </View>

  { /* 课程信息 */ }
  <View style={styles.courseBody}>
    <View style={styles.courseTop}>
      <Text style={styles.courseName} numberOfLines={1}>{classroom.name}</Text>
      <Text style={styles.courseCat}>{category} · 共 {totalSections} 节</Text>
    </View>
    <View style={styles.courseBottom}>
      <Text style={styles.courseTime}>{formatDate(classroom.created_at)}</Text>
      <View style={styles.courseProgress}>
        <View style={styles.progressTrack}>
          <View style={[
            styles.progressFill,
            { width: `${progress}%` },
            status === 'completed' && styles.progressFillComplete
          ]} />
        </View>
        <Text style={styles.progressPct}>{progress}%</Text>
      </View>
      <View style={[styles.courseStatus, { backgroundColor: statusInfo.bgColor }]}>
        <Text style={[styles.courseStatusText, { color: statusInfo.textColor }]}>
          {statusInfo.label}
        </Text>
      </View>
    </View>
  </View>
</Animated.View>
</TouchableOpacity>
);
}

if (loading) {
  return (
    <View style={styles.center}>
      <ActivityIndicator size="large" color={iOSColors.accent} />
    </View>
  );
}

if (error) {
  return (

```

侧伴移动端软件 V1.0.0 第 28 页

```

    <View style={styles.center}>
      <Icons name="alert-circle-outline" size={48} color="#ef4444" />
      <Text style={styles.errorText}>{error}</Text>
      <TouchableOpacity style={styles.retryBtn} onPress={loadClassrooms}>
        <Text style={styles.retryText}>重试</Text>
      </TouchableOpacity>
    </View>
  );
}

return (
  <View style={styles.container}>
    { /* 页面头部 */ }
    <View style={styles.pageHeader}>
      <TouchableOpacity
        style={styles.backBtn}
        onPress={() => router.back()}
        activeOpacity={0.7}
      >
        <Icons name="chevron-back" size={20} color={iOSColors.fg} />

```

```

</TouchableOpacity>
<Text style={styles.pageTitle}>我的课程</Text>
<View style={styles.pageHeaderActions}>
  <TouchableOpacity
    style={styles.headerAction}
    onPress={() => {
      haptics.light();
      onPress();
      // 后续添加搜索功能
    }}
    activeOpacity={0.7}
  >
    <Ionicons name="search-outline" size={18} color={iOSColors.muted} />
  </TouchableOpacity>
</View>
</View>

{/* 筛选标签 */}
<View style={styles.filterTabs}>
  <TouchableOpacity
    style={[styles.filterTab, activeFilter === 'all' && styles.filterTabActive]}
    onPress={() => {
      haptics.light();
      setActiveFilter('all');
    }}
    activeOpacity={0.7}
  >
    <Text style={[styles.filterTabText, activeFilter === 'all' && styles.filterTabTextActive]}>
      全部 <Text style={{ opacity: 0.6 }}>{courseStats.total}</Text>
    </Text>
  </TouchableOpacity>
  <TouchableOpacity
    style={[styles.filterTab, activeFilter === 'progress' && styles.filterTabActive]}
    onPress={() => {
      haptics.light();
      setActiveFilter('progress');
    }}
    activeOpacity={0.7}
  >
    <Text style={[styles.filterTabText, activeFilter === 'progress' && styles.filterTabTextActive]}>
      学习中 <Text style={{ opacity: 0.6 }}>{courseStats.inProgress}</Text>
    </Text>
  </TouchableOpacity>
  <TouchableOpacity
    style={[styles.filterTab, activeFilter === 'completed' && styles.filterTabActive]}
    onPress={() => {
      haptics.light();
      setActiveFilter('completed');
    }}
    activeOpacity={0.7}
  >
    <Text style={[styles.filterTabText, activeFilter === 'completed' && styles.filterTabTextActive]}>
      已完成 <Text style={{ opacity: 0.6 }}>{courseStats.completed}</Text>
    </Text>
  </TouchableOpacity>
  <TouchableOpacity
    style={[styles.filterTab, activeFilter === 'notstarted' && styles.filterTabActive]}
    onPress={() => {
      haptics.light();
      setActiveFilter('notstarted');
    }}
    activeOpacity={0.7}
  >
    <Text style={[styles.filterTabText, activeFilter === 'notstarted' && styles.filterTabTextActive]}>
      未开始 <Text style={{ opacity: 0.6 }}>{courseStats.notStarted}</Text>
    </Text>
  </TouchableOpacity>
</View>

```

```

</TouchableOpacity>
<TouchableOpacity
  style={[styles.filterTab, activeFilter === 'progress' && styles.filterTabActive]}
  onPress={() => {
    haptics.light();
    setActiveFilter('progress');
  }}
  activeOpacity={0.7}
>
  <Text style={[styles.filterTabText, activeFilter === 'progress' && styles.filterTabTextActive]}>
    学习中 <Text style={{ opacity: 0.6 }}>{courseStats.inProgress}</Text>
  </Text>
</TouchableOpacity>
<TouchableOpacity
  style={[styles.filterTab, activeFilter === 'completed' && styles.filterTabActive]}
  onPress={() => {
    haptics.light();
    setActiveFilter('completed');
  }}
  activeOpacity={0.7}
>
  <Text style={[styles.filterTabText, activeFilter === 'completed' && styles.filterTabTextActive]}>
    已完成 <Text style={{ opacity: 0.6 }}>{courseStats.completed}</Text>
  </Text>
</TouchableOpacity>
<TouchableOpacity
  style={[styles.filterTab, activeFilter === 'notstarted' && styles.filterTabActive]}
  onPress={() => {
    haptics.light();
    setActiveFilter('notstarted');
  }}
  activeOpacity={0.7}
>
  <Text style={[styles.filterTabText, activeFilter === 'notstarted' && styles.filterTabTextActive]}>
    未开始 <Text style={{ opacity: 0.6 }}>{courseStats.notStarted}</Text>
  </Text>
</TouchableOpacity>

```

```

    </Text>
  </TouchableOpacity>
</View>

{/* 排序栏 */}
<View style={styles.sortBar}>
  <Text style={styles.courseCount}>共 {filteredClassrooms.length} 门课程</Text>
  <TouchableOpacity
    style={styles.sortBtn}
    onPress={() => {
      haptics.light();
      onPress();
      // 后续添加排序功能
    }}
    activeOpacity={0.7}
  >

```

侧伴移动端软件 V1.0.0 第 30 页

```

    >
    <Text style={styles.sortBtnText}>最近更新</Text>
    <Ionicons name="chevron-down" size={14} color={iOSColors.muted} />
  </TouchableOpacity>
</View>

{/* 课程列表 */}
<FlatList
  data={filteredClassrooms}
  keyExtractor={({item}) => item.id}
  renderItem={({ item, index }) => <CourseCard classroom={item} index={index} />}
  ListEmptyComponent={
    <View style={styles.emptyState}>
      <View style={styles.emptyIcon}>
        <Ionicons name="folder-open-outline" size={28} color={iOSColors.accent} />
      </View>
      <Text style={styles.emptyTitle}>暂无课程</Text>
      <Text style={styles.emptyDesc}>创建你的第一个课程开始学习</Text>
    </View>
  }
  refreshing={loading}
  onRefresh={loadClassrooms}
  contentContainerStyle={styles.courseList}
  showsVerticalScrollIndicator={false}
/>
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: iOSColors.bgSolid,
  },
  // 页面头部
  pageHeader: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: Spacing.sm,
    paddingHorizontal: Spacing.md,
    paddingTop: Spacing.md,
    paddingBottom: Spacing.sm,
  },
  backBtn: {
    width: 44,
    height: 44,
    borderRadius: 22,
    backgroundColor: 'rgba(255, 255, 255, 0.5)',
    alignItems: 'center',
  },

```



```
{ path: '/avatars/teacher.png', desc: '专业的教师形象, 适合主讲老师角色' },
{ path: '/avatars/assist-2.png', desc: '温和的助教形象, 适合辅助教学角色' },
{ path: '/avatars/curious.png', desc: '好奇的学生形象, 适合积极参与的学生角色' },
{ path: '/avatars/thinker.png', desc: '思考型学生形象, 适合深度思考的学生角色' },
{ path: '/avatars/note-taker.png', desc: '记录型学生形象, 适合认真笔记的学生角色' },
{ path: '/avatars/teacher-2.png', desc: '亲切的女教师形象, 适合主讲老师角色' },
{ path: '/avatars/assist.png', desc: '专业的助教形象, 适合辅助教学角色' },
{ path: '/avatars/curious-2.png', desc: '活泼的学生形象, 适合积极互动的学生角色' },
{ path: '/avatars/thinker-2.png', desc: '沉思型学生形象, 适合深度分析的学生角色' },
{ path: '/avatars/note-taker-2.png', desc: '细致的学生形象, 适合记录整理的学生角色' },
] as const;
// ===== 文件: lib/constants/theme.ts =====
// packages/mobile/lib/constants/theme.ts

/**
 * EduDash 品牌设计系统
 * 以橙色为主色调传递活力与热情
 * 参考 packages/mobile/html/index.html 设计
 */

export const Colors = {
  // 主色系 - EduDash 橙色品牌色
  primary: {
    main: '#ec5b13',
    light: '#f97316', // 亮橙 - 悬停状态
    dark: '#ea580c',
    transparent: 'rgba(236, 91, 19, 0.1)',
  },

  // 辅助色系
  secondary: {
    success: '#10B981',
    successLight: '#D1FAE5',
    successBorder: '#6EE7B7',
    info: '#2563EB', // 亮蓝作为信息色
    infoLight: '#DBEAFE',
    infoBorder: '#93C5FD',
    slate: '#64748B', // 石板灰 - 辅助文字
    slateLight: '#F1F5F9',
    slateBorder: '#CBD5E1',
  },

  // 强调色 - 琥珀色 (用于徽章、通知、CTA)
  accent: {
    main: '#F59E0B',
    light: '#FBBF24',
    dark: '#D97706',
  },

  // 背景与中性色
```

```
neutral: {
  background: '#f8f6f6', // EduDash 浅色背景
  backgroundAlt: '#FFFFFF', // 纯白备用背景
  backgroundDark: '#221610', // 深色背景
  card: '#FFFFFF', // 卡片白色
  border: '#f1f5f9', // 浅灰边框
  borderAlt: '#e2e8f0',
  textPrimary: '#0f172a', // 深色文字 (slate-900)
  textSecondary: '#64748b', // 辅助文字 (slate-500)
```

```

    textMuted: '#94a3b8',      // 淡化文字 (slate-400)
    textInverse: '#FFFFFF',    // 反色文字 (深色背景上)
    white: '#FFFFFF',
    disabled: '#e2e8f0',
    disabledText: '#94a3b8',
  },

  // 语义颜色 - 用于统计卡片、功能入口等
  semantic: {
    blue: '#2563eb',          // 学生/信息
    orange: '#f59e0b',        // 时间/作业
    green: '#10b981',         // 成功/续费
    purple: '#8b5cf6',         // 损失指标
    red: '#ef4444',           // 紧急/考试
    teal: '#14b8a6',          // 课程
    indigo: '#6366f1',        // 考勤
    pink: '#ec4899',          // 性能监控
    amber: '#f59e0b',         // 报告/进行中
  },

  // 结果反馈色
  feedback: {
    successBg: '#D1FAE5',
    successBorder: '#6EE7B7',
    successText: '#059669',
    errorBg: '#FEF2F2',
    errorBorder: '#FECACA',
    errorText: '#B91C1C',
    warningBg: '#FFFBE5',
    warningBorder: '#FDE68A',
    warningText: '#B45309',
  },

  // 阴影色 (扁平化设计不使用阴影, 仅保留透明度用于交互反馈)
  shadow: {
    primary: 'rgba(30, 64, 175, 0.1)',
    success: 'rgba(16, 185, 129, 0.1)',
    info: 'rgba(37, 99, 235, 0.1)',
    accent: 'rgba(245, 158, 11, 0.1)',
    neutral: 'rgba(0, 0, 0, 0.02)',
  },
},

```

侧伴移动端软件 V1.0.0 第 33 页

```

};

// 圆角规范 - EduDash 设计
export const Rounded = {
  sm: 8,      // 小按钮、输入框 (xl in Tailwind)
  md: 12,     // 卡片 (2xl in Tailwind)
  lg: 16,     // 大卡片/功能按钮
  xl: 24,     // 特大圆角
  full: 9999, // 徽章/标签 (胶囊形)
};

// 间距规范 (4px基准网格)
export const Spacing = {
  xs: 4,
  sm: 8,
  md: 16,
  lg: 24,
  xl: 32,
  xxl: 48,
};

// 字体规范
export const Typography = {
  fontFamily: 'Inter, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif',

```

```

h1: {
  fontSize: 40,
  fontWeight: '700',
  lineHeight: 1.2,
  letterSpacing: -0.02,
},
h2: {
  fontSize: 28,
  fontWeight: '600',
  lineHeight: 1.3,
  letterSpacing: -0.01,
},
h3: {
  fontSize: 20,
  fontWeight: '600',
  lineHeight: 1.4,
},
bodyLg: {
  fontSize: 18,
  fontWeight: '400',
  lineHeight: 1.7,
},
bodyMd: {
  fontSize: 16,
  fontWeight: '400',
  lineHeight: 1.6,

```

侧伴移动端软件 V1.0.0 第 34 页

```

},
bodySm: {
  fontSize: 14,
  fontWeight: '400',
  lineHeight: 1.5,
},
label: {
  fontSize: 12,
  fontWeight: '500',
  lineHeight: 1.4,
  letterSpacing: 0.05,
},
};

// 预设的辅助色映射（用于功能图标等） - EduDash 语义颜色
export const SecondaryColorMap: Record<string, string> = {
  courses: Colors.semantic.teal,      // 课程 - 青色
  questions: Colors.semantic.blue,    // 问答 - 蓝色
  notes: Colors.semantic.green,       // 笔记 - 绿色
  buddy: Colors.semantic.orange,      // 学习搭子 - 橙色
  matching: Colors.semantic.indigo,   // 学习匹配 - 紫蓝色
  gamification: Colors.primary.main,  // 成长体系 - 主色
  invite: Colors.semantic.pink,       // 邀请奖励 - 粉色
  payment: Colors.semantic.amber,     // 充值中心 - 琥珀色
  wallet: Colors.semantic.purple,     // 钱包 - 紫色
  enterprise: Colors.semantic.blue,   // 企业服务 - 蓝色
  knowledge: Colors.semantic.teal,    // 知识 - 青色
  class: Colors.primary.main,         // 课堂 - 主色
  student: Colors.semantic.blue,     // 学生 - 蓝色
  homework: Colors.semantic.orange,   // 作业 - 橙色
  exam: Colors.semantic.red,          // 考试 - 红色
  performance: Colors.semantic.pink,  // 性能 - 粉色
  report: Colors.semantic.amber,      // 报告 - 琥珀色
  attend: Colors.semantic.indigo,     // 考勤 - 紫蓝色
};

// 获取对应颜色的透明背景
export function getShadowColor(color: string): string {
  if (color === Colors.primary.main) return Colors.shadow.primary;

```

```
if (color === Colors.secondary.success) return Colors.shadow.success;
if (color === Colors.secondary.info) return Colors.shadow.info;
if (color === Colors.accent.main) return Colors.shadow.accent;
return Colors.shadow.neutral;
}

// 按钮样式预设
export const ButtonStyles = {
  primary: {
    backgroundColor: Colors.primary.main,
    textColor: Colors.neutral.textInverse,
```

侧伴移动端软件 V1.0.0 第 35 页

```
    borderRadius: Rounded.sm,
    paddingVertical: 12,
    paddingHorizontal: 24,
  },
  secondary: {
    backgroundColor: Colors.neutral.backgroundAlt,
    textColor: Colors.primary.main,
    borderRadius: Rounded.sm,
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderWidth: 1,
    borderColor: Colors.primary.main,
  },
  accent: {
    backgroundColor: Colors.accent.main,
    textColor: Colors.neutral.textInverse,
    borderRadius: Rounded.sm,
    paddingVertical: 12,
    paddingHorizontal: 24,
  },
};

// 卡片样式预设
export const CardStyles = {
  default: {
    backgroundColor: Colors.neutral.card,
    borderRadius: Rounded.md,
    padding: Spacing.lg,
    borderWidth: 1,
    borderColor: Colors.neutral.border,
  },
  elevated: {
    backgroundColor: Colors.neutral.card,
    borderRadius: Rounded.md,
    padding: Spacing.lg,
    borderWidth: 1,
    borderColor: Colors.neutral.border,
  },
};

// 输入框样式预设
export const InputStyles = {
  default: {
    backgroundColor: Colors.neutral.backgroundAlt,
    borderRadius: Rounded.sm,
    paddingVertical: 10,
    paddingHorizontal: 14,
    borderWidth: 1,
    borderColor: Colors.neutral.border,
    textColor: Colors.neutral.textPrimary,
```

```

    },
  };

// 徽章样式预设
export const BadgeStyles = {
  default: {
    backgroundColor: Colors.accent.main,
    textColor: Colors.neutral.textInverse,
    borderRadius: Rounded.full,
    paddingVertical: 4,
    paddingHorizontal: 12,
  },
  success: {
    backgroundColor: Colors.secondary.success,
    textColor: Colors.neutral.textInverse,
    borderRadius: Rounded.full,
    paddingVertical: 4,
    paddingHorizontal: 12,
  },
  info: {
    backgroundColor: Colors.secondary.info,
    textColor: Colors.neutral.textInverse,
    borderRadius: Rounded.full,
    paddingVertical: 4,
    paddingHorizontal: 12,
  },
};

// ===== 文件: lib/hooks/use-classrooms.ts =====
import { useState, useEffect } from 'react';
import { apiClient } from '@lib/api-client';

interface Classroom {
  id: string;
  name: string;
  description?: string;
  language_directive?: string;
  created_at: string;
  updated_at: string;
}

export function useClassrooms() {
  const [classrooms, setClassrooms] = useState<Classroom[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  async function refresh() {
    setLoading(true);
    setError(null);
    try {
      const data = await apiClient.getClassrooms();

```

```

      setClassrooms(data);
    } catch (err) {
      setError(err instanceof Error ? err.message : '加载失败');
    } finally {
      setLoading(false);
    }
  }

  return { classrooms, loading, error, refresh };
}

// ===== 文件: lib/hooks/use-feedback.ts =====
// packages/mobile/lib/hooks/use-feedback.ts

```

```

import { useCallback } from 'react';
import * as Haptics from 'expo-haptics';

/**
 * 交互反馈 Hook
 * 提供统一的触觉反馈和视觉反馈触发函数
 */

export type FeedbackStyle = 'light' | 'medium' | 'heavy';
export type NotificationType = 'success' | 'warning' | 'error';

export function useFeedback() {
  // 点击反馈
  const onTap = useCallback((style: FeedbackStyle = 'light') => {
    const hapticStyle = style === 'light'
      ? Haptics.ImpactFeedbackStyle.Light
      : style === 'medium'
      ? Haptics.ImpactFeedbackStyle.Medium
      : Haptics.ImpactFeedbackStyle.Heavy;

    Haptics.impactAsync(hapticStyle);
  }, []);

  // 按钮点击（默认轻触觉）
  const onPress = useCallback(() => {
    onTap('light');
  }, [onTap]);

  // 重要操作（中等触觉）
  const onImportant = useCallback(() => {
    onTap('medium');
  }, [onTap]);

  // 结果通知
  const onNotify = useCallback((type: NotificationType) => {
    const notificationType = type === 'success'
      ? Haptics.NotificationFeedbackType.Success

```

侧伴移动端软件 V1.0.0 第 38 页

```

      : type === 'error'
      ? Haptics.NotificationFeedbackType.Error
      : Haptics.NotificationFeedbackType.Warning;

    Haptics.notificationAsync(notificationType);
  }, []);

  // 成功反馈
  const onSuccess = useCallback(() => {
    onNotify('success');
  }, [onNotify]);

  // 错误反馈
  const onError = useCallback(() => {
    onNotify('error');
  }, [onNotify]);

  // 警告反馈
  const onWarning = useCallback(() => {
    onNotify('warning');
  }, [onNotify]);

  // 选择反馈（用于选项切换）
  const onSelection = useCallback(() => {
    Haptics.selectionAsync();
  }, []);

```

```

    return {
      onTap,
      onPress,
      onImportant,
      onNotify,
      onSuccess,
      onError,
      onWarning,
      onSelection,
    };
  }
}
// ===== 文件: lib/hooks/use-first-time-hint.ts =====
/**
 * useFirstTimeHint -- 一次性操作提示控制
 *
 * 使用 AsyncStorage 记忆用户是否已看过某个提示，避免重复打扰。
 *
 * 用法:
 *   const { visible, dismiss } = useFirstTimeHint('courses.longPress');
 *   return visible ? <HintToast onClose={dismiss} ... /> : null;
 */
import { useCallback, useEffect, useState } from 'react';
import AsyncStorage from '@react-native-async-storage/async-storage';

```

侧伴移动端软件 V1.0.0 第 39 页

```

const STORAGE_PREFIX = 'hint_seen:';

export interface UseFirstTimeHintOptions {
  /** 是否启用（默认 true, false 时永不显示） */
  enabled?: boolean;
  /** 延迟多少毫秒后再显示（默认 400ms, 避免与页面进入动画重叠） */
  delayMs?: number;
  /** 若 > 0, 显示后自动消失的时长（毫秒）。默认 0 表示不自动消失 */
  autoHideMs?: number;
}

export function useFirstTimeHint(key: string, options: UseFirstTimeHintOptions = {}) {
  const { enabled = true, delayMs = 400, autoHideMs = 0 } = options;
  const [visible, setVisible] = useState(false);

  const storageKey = `${STORAGE_PREFIX}${key}`;

  const dismiss = useCallback(() => {
    setVisible(false);
    AsyncStorage.setItem(storageKey, '1').catch(() => {
      /* 记录失败不影响当前体验 */
    });
  }, [storageKey]);

  useEffect(() => {
    if (!enabled) return;

    let cancelled = false;
    let showTimer: ReturnType<typeof setTimeout> | null = null;
    let hideTimer: ReturnType<typeof setTimeout> | null = null;

    (async () => {
      try {
        const seen = await AsyncStorage.getItem(storageKey);
        if (cancelled || seen) return;

        showTimer = setTimeout(() => {
          if (cancelled) return;
          setVisible(true);
          if (autoHideMs > 0) {
            hideTimer = setTimeout(() => {

```

```

        if (cancelled) return;
        setVisible(false);
        AsyncStorage.setItem(storageKey, '1').catch(() => {});
    }, autoHideMs);
}
}, delayMs);
} catch {
    /* 读失败就当未看过，下次重试 */

```

侧伴移动端软件 V1.0.0 第 40 页

```

    }
  })();

  return () => {
    cancelled = true;
    if (showTimer) clearTimeout(showTimer);
    if (hideTimer) clearTimeout(hideTimer);
  };
}, [enabled, storageKey, delayMs, autoHideMs]);

return { visible, dismiss };
}

/** 调试用：清除指定 key（或全部）的首次提示记忆 */
export async function resetFirstTimeHint(key?: string): Promise<void> {
  if (key) {
    await AsyncStorage.removeItem(`${STORAGE_PREFIX}${key}`);
    return;
  }
  const allKeys = await AsyncStorage.getAllKeys();
  const hintKeys = allKeys.filter((k) => k.startsWith(STORAGE_PREFIX));
  if (hintKeys.length > 0) {
    await AsyncStorage.multiRemove(hintKeys);
  }
}

// ===== 文件: lib/hooks/use-haptics.ts =====
import * as Haptics from 'expo-haptics';

/**
 * 统一的触觉反馈 Hook
 * 提供一致的触觉反馈体验
 */
export const useHaptics = () => {
  const light = () => Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
  const medium = () => Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium);
  const heavy = () => Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Heavy);
  const success = () => Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
  const warning = () => Haptics.notificationAsync(Haptics.NotificationFeedbackType.Warning);
  const error = () => Haptics.notificationAsync(Haptics.NotificationFeedbackType.Error);

  return { light, medium, heavy, success, warning, error };
};

// ===== 文件: lib/i18n/index.ts =====
/**
 * 移动端国际化支持
 */

export type Locale = 'zh-CN' | 'en-US';

```

侧伴移动端软件 V1.0.0 第 41 页

```

export const defaultLocale: Locale = 'zh-CN';

```



```
const translations = {
  'zh-CN': {
    // 通用
    common: {
      loading: '加载中...',
      error: '出错了',
      retry: '重试',
      cancel: '取消',
      confirm: '确定',
      save: '保存',
      delete: '删除',
      edit: '编辑',
      back: '返回',
      next: '下一步',
      previous: '上一步',
      done: '完成',
    },
    // 认证
    auth: {
      login: '登录',
      register: '注册',
      email: '邮箱',
      password: '密码',
      nickname: '昵称',
      loginSuccess: '登录成功',
      loginFailed: '登录失败',
      registerSuccess: '注册成功',
      registerFailed: '注册失败',
      logout: '退出登录',
      agreePolicy: '我已阅读并同意用户协议和隐私政策',
    },
    // 课程
    classroom: {
      title: '我的课程',
      create: '创建课程',
      delete: '删除课程',
      noClassrooms: '暂无课程',
      loadingScene: '加载场景...',
      sceneProgress: '场景 {current} / {total}',
      generating: '生成中...',
      searchPlaceholder: '搜索课程...',
      noResults: '未找到匹配的课程',
      searchHint: '尝试其他关键词',
      // 新增iOS风格页面的翻译
      all: '全部',
      inProgress: '学习中',
      completed: '已完成',
      notStarted: '未开始',

```

侧伴移动端软件 V1.0.0 第 42 页

```
recentCourses: '近期课程',
recommendedCourses: '推荐课程',
courseCatalog: '课程目录',
courseIntro: '课程简介',
expandAll: '展开全部',
collapse: '收起',
continueLearning: '继续学习',
download: '下载',
share: '分享',
totalLessons: '共 {count} 节',
lessons: '课时',
totalDuration: '总时长',
difficulty: '难度',
students: '学员',
rating: '评分',
current: '当前',

```

```
    slide: '幻灯片',
    quiz: '测验',
    interactive: '互动',
    scene: '场景',
  },
  // 白板
  whiteboard: {
    title: '互动白板',
    open: '打开白板',
    close: '关闭白板',
    clear: '清空',
    draw: '绘制',
    text: '文字',
    shape: '形状',
    color: '颜色',
    strokeWidth: '笔触宽度',
    undo: '撤销',
    redo: '重做',
    history: '历史',
  },
  // 激光笔/聚光灯
  pointer: {
    laser: '激光笔',
    spotlight: '聚光灯',
    enable: '启用',
    disable: '关闭',
    followTouch: '跟随触摸',
  },
  // 测验
  quiz: {
    title: '随堂测验',
    start: '开始答题',
    submit: '提交答案',
    correct: '正确',
```

侧伴移动端软件 V1.0.0 第 43 页

```
    incorrect: '错误',
    score: '得分',
    analysis: '解析',
    singleChoice: '单选题',
    multipleChoice: '多选题',
    shortAnswer: '简答题',
    aiGrading: 'AI 正在批改...',
    retry: '重新答题',
    nextQuestion: '下一题',
    prevQuestion: '上一题',
    swipeHint: '左右滑动切换题目',
  },
  // Agent
  agent: {
    teacher: '教师',
    assistant: '助教',
    student: '学生',
    thinking: '思考中...',
    speaking: '正在讲解',
    askQuestion: '提问',
    voiceInput: '语音输入',
    textInput: '文字输入',
  },
  // 聊天
  chat: {
    placeholder: '输入消息...',
    send: '发送',
    recording: '录音中...',
    stopRecording: '停止录音',
    listening: '正在聆听...',
  },
},
```

```
// 导航
navigation: {
  swipeLeft: '左滑下一页',
  swipeRight: '右滑上一页',
  pinchZoom: '双指缩放',
  doubleTap: '双击全屏',
},
// 成就
achievement: {
  earned: '已获得',
  progress: '进度',
  points: '积分',
  newAchievement: '恭喜获得新成就! ',
},
// 课程完成
classroomComplete: {
  title: '课程完成',
  trailLabels: {
    slide: '页',
```

侧伴移动端软件 V1.0.0 第 44 页

```
    quiz: '小测',
    interactive: '互动',
    pbl: '项目',
  },
  quizScoreLabel: '答对 {{correct}} / {{{total}}}',
  encouragement: {
    high: '太棒了, 完美发挥! ',
    mid: '表现不错, 继续加油! ',
    low: '万事开头难, 回去再练练吧。',
  },
},
// 首页
home: {
  title: '首页',
  greetingMorning: '早上好',
  greetingAfternoon: '下午好',
  greetingEvening: '晚上好',
  greetingNight: '夜深了',
  streakDays: '天连续学习',
  days: '天',
  courses: '门课程',
  hours: '小时',
  myCourses: '我的课程',
  qaBounty: '问答悬赏',
  sharedNotes: '共享笔记',
  studyBuddy: '学习搭子',
  studyMatching: '学习匹配',
  growthSystem: '成长体系',
  inviteRewards: '邀请奖励',
  recharge: '充值中心',
  wallet: '钱包',
  enterpriseServices: '企业服务',
  allNotes: '全部笔记',
  favorites: '收藏',
  today: '今日',
  quickRecord: '快速记录',
  notes: '我的笔记',
  streak: '连续学习',
  notification: '通知',
  newLessonReminder: '新课提醒',
  startLearning: '开始学习',
  remindLater: '稍后提醒',
},
checkin: {
  today: '今日打卡',
  streak: '连续 {days} 天',
```

```
    alreadyChecked: '今日已打卡',
    checkinSuccess: '打卡成功! ',
  },
  // 发现
```

侧伴移动端软件 V1.0.0 第 45 页

```
discover: {
  title: '发现',
  popular: '热门课程',
  recent: '最新分享',
  likes: '{count} 人喜欢',
  views: '{count} 次浏览',
},
// 个人
profile: {
  title: '我的',
  settings: '设置',
  language: '语言',
  about: '关于',
  exportData: '导出数据',
  deleteAccount: '注销账号',
  statistics: '学习统计',
  // 新增
  languageSettings: '语言设置',
  chinese: '简体中文',
  english: 'English',
  tokenBalance: 'Token余额',
  pointsBalance: '积分余额',
},
// 设置
settings: {
  language: '语言设置',
  theme: '主题',
  notifications: '通知',
  autoPlay: '自动播放',
  playbackSpeed: '播放速度',
},
// 错误消息
errors: {
  networkError: '网络连接失败',
  serverError: '服务器错误',
  unauthorized: '请先登录',
  forbidden: '无权限访问',
  notFound: '未找到内容',
  validationError: '输入有误',
},
},
'en-US': {
  common: {
    loading: 'Loading...',
    error: 'Error',
    retry: 'Retry',
    cancel: 'Cancel',
    confirm: 'Confirm',
    save: 'Save',
    delete: 'Delete',
```

侧伴移动端软件 V1.0.0 第 46 页

```
edit: 'Edit',
back: 'Back',
next: 'Next',
previous: 'Previous',
done: 'Done',
```

```

},
auth: {
  login: 'Login',
  register: 'Register',
  email: 'Email',
  password: 'Password',
  nickname: 'Nickname',
  loginSuccess: 'Login successful',
  loginFailed: 'Login failed',
  registerSuccess: 'Registration successful',
  registerFailed: 'Registration failed',
  logout: 'Logout',
  agreePolicy: 'I agree to the User Agreement and Privacy Policy',
},
// 课程
classroom: {
  title: 'My Classrooms',
  create: 'Create Classroom',
  delete: 'Delete Classroom',
  noClassrooms: 'No classrooms',
  loadingScene: 'Loading scene...',
  sceneProgress: 'Scene {current} / {total}',
  generating: 'Generating...',
  searchPlaceholder: 'Search classrooms...',
  noResults: 'No matching classrooms',
  searchHint: 'Try different keywords',
  // iOS-style pages
  all: 'All',
  inProgress: 'In Progress',
  completed: 'Completed',
  notStarted: 'Not Started',
  recentCourses: 'Recent Courses',
  recommendedCourses: 'Recommended',
  courseCatalog: 'Course Catalog',
  courseIntro: 'Introduction',
  expandAll: 'Expand All',
  collapse: 'Collapse',
  continueLearning: 'Continue Learning',
  download: 'Download',
  share: 'Share',
  totalLessons: '{count} lessons',
  lessons: 'Lessons',
  totalDuration: 'Duration',
  difficulty: 'Level',
  students: 'Students',

```

```

  rating: 'Rating',
  current: 'Current',
  slide: 'Slide',
  quiz: 'Quiz',
  interactive: 'Interactive',
  scene: 'Scene',
},
whiteboard: {
  title: 'Interactive Whiteboard',
  open: 'Open Whiteboard',
  close: 'Close Whiteboard',
  clear: 'Clear',
  draw: 'Draw',
  text: 'Text',
  shape: 'Shape',
  color: 'Color',
  strokeWidth: 'Stroke Width',
  undo: 'Undo',
  redo: 'Redo',
  history: 'History',

```

```

},
pointer: {
  laser: 'Laser Pointer',
  spotlight: 'Spotlight',
  enable: 'Enable',
  disable: 'Disable',
  followTouch: 'Follow Touch',
},
quiz: {
  title: 'Quiz',
  start: 'Start Quiz',
  submit: 'Submit Answers',
  correct: 'Correct',
  incorrect: 'Incorrect',
  score: 'Score',
  analysis: 'Analysis',
  singleChoice: 'Single Choice',
  multipleChoice: 'Multiple Choice',
  shortAnswer: 'Short Answer',
  aiGrading: 'AI is grading...',
  retry: 'Retry',
  nextQuestion: 'Next',
  prevQuestion: 'Previous',
  swipeHint: 'Swipe left/right to change question',
},
agent: {
  teacher: 'Teacher',
  assistant: 'Assistant',
  student: 'Student',
  thinking: 'Thinking...',

```

```

  speaking: 'Speaking',
  askQuestion: 'Ask a question',
  voiceInput: 'Voice Input',
  textInput: 'Text Input',
},
chat: {
  placeholder: 'Type a message...',
  send: 'Send',
  recording: 'Recording...',
  stopRecording: 'Stop Recording',
  listening: 'Listening...',
},
navigation: {
  swipeLeft: 'Swipe left for next',
  swipeRight: 'Swipe right for previous',
  pinchZoom: 'Pinch to zoom',
  doubleTap: 'Double tap for fullscreen',
},
achievement: {
  earned: 'Earned',
  progress: 'Progress',
  points: 'Points',
  newAchievement: 'New achievement unlocked!',
},
classroomComplete: {
  title: 'Course Complete',
  traillabels: {
    slide: 'Slides',
    quiz: 'Quizzes',
    interactive: 'Interactive',
    pbl: 'Projects',
  },
  quizScoreLabel: '{{correct}} / {{total}} correct',
  encouragement: {
    high: 'Excellent, perfect score!',

```

```

        mid: 'Good job, keep it up!',
        low: 'Keep practicing, you\'ll get better!',
    },
},
checkin: {
    today: 'Today\'s Check-in',
    streak: '{days} day streak',
    alreadyChecked: 'Already checked in today',
    checkinSuccess: 'Check-in successful!',
},
discover: {
    title: 'Discover',
    popular: 'Popular',
    recent: 'Recent',
    likes: '{count} likes',

```

侧伴移动端软件 V1.0.0 第 49 页

```

        views: '{count} views',
    },
    profile: {
        title: 'Me',
        settings: 'Settings',
        language: 'Language',
        about: 'About',
        exportData: 'Export Data',
        deleteAccount: 'Delete Account',
        statistics: 'Statistics',
        // New
        languageSettings: 'Language Settings',
        chinese: 'Chinese',
        english: 'English',
        tokenBalance: 'Token Balance',
        pointsBalance: 'Points Balance',
    },
    settings: {
        language: 'Language',
        theme: 'Theme',
        notifications: 'Notifications',
        autoPlay: 'Auto-play',
        playbackSpeed: 'Playback Speed',
    },
    errors: {
        networkError: 'Network connection failed',
        serverError: 'Server error',
        unauthorized: 'Please login first',
        forbidden: 'Access denied',
        notFound: 'Not found',
        validationError: 'Invalid input',
    },
},
};

class I18n {
    private locale: Locale = defaultLocale;

    setLocale(locale: Locale) {
        this.locale = locale;
    }

    getLocale(): Locale {
        return this.locale;
    }
}

/**
 * 获取翻译文本
 * @param key - 翻译键, 如 'common.loading'
 * @param params - 可选参数, 用于替换 {xxx}

```

```

    */
    t(key: string, params?: Record<string, string | number>): string {
        const keys = key.split('.');
        let value: unknown = translations[this.locale];

        for (const k of keys) {
            value = (value as Record<string, unknown>)?.[k];
        }

        if (typeof value !== 'string') {
            // 尝试获取默认语言的翻译
            let fallbackValue: unknown = translations[defaultLocale];
            for (const k of keys) {
                fallbackValue = (fallbackValue as Record<string, unknown>)?.[k];
            }
            value = fallbackValue;
        }

        if (typeof value !== 'string') {
            return key;
        }

        // 替换参数
        if (params) {
            let result = value;
            for (const [paramKey, paramValue] of Object.entries(params)) {
                result = result.replace(`${paramKey}`, String(paramValue));
            }
            return result;
        }

        return value;
    }

    /**
     * 获取所有可用语言
     */
    getAvailableLocales(): { code: Locale; name: string }[] {
        return [
            { code: 'zh-CN', name: '简体中文' },
            { code: 'en-US', name: 'English (US)' },
        ];
    }
}

export const i18n = new I18n();

// React Hook
import { useState, useEffect } from 'react';
import * as AsyncStorage from 'expo-secure-store';

```

```

export function useI18n() {
    const [locale, setLocaleState] = useState<Locale>(defaultLocale);

    useEffect(() => {
        loadSavedLocale();
    }, []);

    const loadSavedLocale = async () => {

```



```

    try {
      const saved = await AsyncStorage.getItemAsync('locale');
      if (saved === 'zh-CN' || saved === 'en-US') {
        i18n.setLocale(saved);
        setLocaleState(saved);
      }
    } catch {
      // AsyncStorage unavailable
    }
  };

  const setLocale = async (newLocale: Locale) => {
    i18n.setLocale(newLocale);
    setLocaleState(newLocale);
    try {
      await AsyncStorage.setItemAsync('locale', newLocale);
    } catch {
      // AsyncStorage unavailable
    }
  };

  return {
    t: i18n.t.bind(i18n),
    locale,
    setLocale,
    availableLocales: i18n.getAvailableLocales(),
  };
}

// ===== 文件: lib/quiz/persistence.ts =====
/**
 * Quiz 状态持久化 - 移动端版本
 *
 * 使用 AsyncStorage 存储, 支持断点续答
 */

import AsyncStorage from '@react-native-async-storage/async-storage';

export const DRAFT_KEY_PREFIX = 'quizDraft:';
export const ANSWERS_KEY_PREFIX = 'quizAnswers:';
export const RESULTS_KEY_PREFIX = 'quizResults:';

```

侧伴移动端软件 V1.0.0 第 52 页

```

export type QuizAnswers = Record<string, string | string[]>;

export interface QuestionResult {
  questionId: string;
  correct: boolean | null; // null = pending grading
  feedback?: string;
}

export type SubmittedState =
  | { kind: 'reviewing'; answers: QuizAnswers; results: QuestionResult[] }
  | { kind: 'answering'; answers: QuizAnswers }
  | null;

export function draftKey(sceneId: string): string {
  return DRAFT_KEY_PREFIX + sceneId;
}

export function answersKey(sceneId: string): string {
  return ANSWERS_KEY_PREFIX + sceneId;
}

export function resultsKey(sceneId: string): string {
  return RESULTS_KEY_PREFIX + sceneId;
}

```

```

/** 读取草稿答案 */
export async function readDraft(sceneId: string): Promise<QuizAnswers> {
  try {
    const raw = await AsyncStorage.getItem(draftKey(sceneId));
    if (raw) {
      return JSON.parse(raw) as QuizAnswers;
    }
  } catch (error) {
    console.warn(`[persistence] readDraft failed for ${sceneId}:`, error);
  }
  return {};
}

/** 写入草稿答案 */
export async function writeDraft(sceneId: string, answers: QuizAnswers): Promise<void> {
  try {
    await AsyncStorage.setItem(draftKey(sceneId), JSON.stringify(answers));
  } catch (error) {
    console.warn(`[persistence] writeDraft failed for ${sceneId}:`, error);
  }
}

/** 清除草稿答案 */
export async function clearDraft(sceneId: string): Promise<void> {
  try {

```

侧伴移动端软件 V1.0.0 第 53 页

```

    await AsyncStorage.removeItem(draftKey(sceneId));
  } catch {
    // ignore
  }
}

/** 读取提交后的状态 */
export async function readSubmittedState(sceneId: string): Promise<SubmittedState> {
  try {
    const rawA = await AsyncStorage.getItem(answersKey(sceneId));
    if (!rawA) return null;

    const answers = JSON.parse(rawA) as QuizAnswers;
    const rawR = await AsyncStorage.getItem(resultsKey(sceneId));

    if (rawR) {
      const results = JSON.parse(rawR) as QuestionResult[];
      if (Array.isArray(results) && results.length > 0) {
        return { kind: 'reviewing', answers, results };
      }
    }
    return { kind: 'answering', answers };
  } catch {
    return null;
  }
}

/** 读取用于汇总的答案（优先提交答案，fallback 到草稿） */
export async function readAnswersForSummary(sceneId: string): Promise<QuizAnswers> {
  try {
    const rawA = await AsyncStorage.getItem(answersKey(sceneId));
    if (rawA) {
      return JSON.parse(rawA) as QuizAnswers;
    }
    const rawD = await AsyncStorage.getItem(draftKey(sceneId));
    if (rawD) {
      return JSON.parse(rawD) as QuizAnswers;
    }
  } catch {

```

```

    // ignore
  }
  return {};
}

/** 提交时写入答案 */
export async function writeSubmittedAnswers(sceneId: string, answers: QuizAnswers): Promise<void> {
  try {
    await AsyncStorage.setItem(answersKey(sceneId), JSON.stringify(answers));
    // 清除草稿
    await AsyncStorage.removeItem(draftKey(sceneId));
  }

```

侧伴移动端软件 V1.0.0 第 54 页

```

  } catch {
    // ignore
  }
}

/** 批改后写入结果 */
export async function writeSubmittedResults(sceneId: string, results: QuestionResult[]): Promise<void> {
  try {
    await AsyncStorage.setItem(resultsKey(sceneId), JSON.stringify(results));
  } catch {
    // ignore
  }
}

/** 重试时清除提交状态 */
export async function clearSubmitted(sceneId: string): Promise<void> {
  try {
    await AsyncStorage.removeItem(answersKey(sceneId));
    await AsyncStorage.removeItem(resultsKey(sceneId));
  } catch {
    // ignore
  }
}

/** 删除场景时清除所有相关数据 */
export async function clearAllForScene(sceneId: string): Promise<void> {
  try {
    await AsyncStorage.multiRemove([
      draftKey(sceneId),
      answersKey(sceneId),
      resultsKey(sceneId),
    ]);
  } catch {
    // ignore
  }
}

/** 批量读取多个场景的答案 */
export async function readAllAnswersForSummary(sceneIds: string[]): Promise<Record<string, QuizAnswers>> {
  const result: Record<string, QuizAnswers> = {};
  for (const sceneId of sceneIds) {
    result[sceneId] = await readAnswersForSummary(sceneId);
  }
  return result;
}

// ===== 文件: lib/storage/audio-storage.ts =====
/**
 * Audio Storage - 使用 expo-file-system 存储 TTS 音频文件
 */

```

```

import { Paths, Directory, File } from 'expo-file-system';

const AUDIO_DIR_NAME = 'audio';

/**
 * 获取音频存储目录
 */
function getAudioDirectory(): Directory {
  return new Directory(Paths.document, AUDIO_DIR_NAME);
}

/**
 * 初始化音频存储目录
 */
export function initAudioStorage(): void {
  const audioDir = getAudioDirectory();
  if (!audioDir.exists) {
    audioDir.create({ idempotent: true });
  }
}

/**
 * 保存音频文件（同步版本，适用于小文件）
 *
 * @param audioId - 音频 ID (不含扩展名)
 * @param base64 - base64 编码的音频数据
 * @param format - 音频格式 (mp3, wav, etc.)
 * @returns 文件路径
 */
export function saveAudioFile(
  audioId: string,
  base64: string,
  format: string = 'mp3'
): string {
  initAudioStorage();

  const audioDir = getAudioDirectory();
  const fileName = `${audioId}.${format}`;

  // Create file and write base64 data
  const file = audioDir.createFile(fileName, `audio/${format}`);
  file.write(base64, { encoding: 'base64' });

  return file.uri;
}

/**
 * 保存音频文件（异步版本，适用于大文件，避免阻塞 UI）
 *
 * @param audioId - 音频 ID (不含扩展名)

```

```

 * @param base64 - base64 编码的音频数据
 * @param format - 音频格式 (mp3, wav, etc.)
 * @returns Promise<string> 文件路径
 */
export async function saveAudioFileAsync(
  audioId: string,
  base64: string,
  format: string = 'mp3'
): Promise<string> {
  // 使用 setTimeout 将同步操作推迟到下一个事件循环，避免阻塞 UI
  return new Promise((resolve, reject) => {
    setTimeout(() => {

```

```

        try {
            const filePath = saveAudioFile(audioId, base64, format);
            resolve(filePath);
        } catch (err) {
            reject(err);
        }
    }, 0);
    });
}

/**
 * 获取音频文件路径
 *
 * @param audioId - 音频 ID
 * @param format - 音频格式 (默认 mp3)
 * @returns 文件路径 (如果不存在返回 null)
 */
export function getAudioPath(
    audioId: string,
    format: string = 'mp3'
): string | null {
    const audioDir = getAudioDirectory();
    const fileName = `${audioId}.${format}`;
    const file = new File(audioDir, fileName);

    if (file.exists) {
        return file.uri;
    }
    return null;
}

/**
 * 删除音频文件
 *
 * @param audioId - 音频 ID
 * @param format - 音频格式
 */
export function deleteAudioFile(

```

侧伴移动端软件 V1.0.0 第 57 页

```

    audioId: string,
    format: string = 'mp3'
): void {
    const audioDir = getAudioDirectory();
    const fileName = `${audioId}.${format}`;
    const file = new File(audioDir, fileName);

    if (file.exists) {
        file.delete();
    }
}

/**
 * 清理所有音频文件
 */
export function clearAllAudioFiles(): void {
    const audioDir = getAudioDirectory();
    if (audioDir.exists) {
        audioDir.delete();
    }
    initAudioStorage();
}

/**
 * 获取所有已存储的音频 ID 列表
 */
export function listStoredAudioIds(): string[] {

```

```

initAudioStorage();

const audioDir = getAudioDirectory();
const files = audioDir.list();

// 提取 audioId (去掉扩展名)
return files
  .filter(f => f instanceof File && (f.name.endsWith('.mp3') || f.name.endsWith('.wav')))
  .map(f => (f as File).name.replace(/\.(\mp3\|wav)$/, ''));
}
// ===== 文件: lib/utils/alert.ts =====
/**
 * 跨平台 Alert 工具
 *
 * Web 端用 window.alert/confirm, Mobile 端用 React Native Alert。
 * 用于替代散落在各业务文件中的 showAlert 重复函数。
 */
import { Platform } from 'react-native';

export interface AlertButton {
  text: string;
  onPress?: () => void;
  style?: 'default' | 'cancel' | 'destructive';
}

```

侧伴移动端软件 V1.0.0 第 58 页

```

}

/**
 * 通用弹窗提示
 * @param title 标题
 * @param message 正文
 * @param buttons 按钮列表 (mobile 原生支持; web 仅触发第一个非 cancel 按钮的 onPress)
 */
export function showAlert(title: string, message: string, buttons?: AlertButton[]): void {
  if (Platform.OS === 'web') {
    if (buttons && buttons.length > 0) {
      const confirmed = window.confirm(`${title}\n\n${message}`);
      if (confirmed) {
        const primary = buttons.find((b) => b.style !== 'cancel') ?? buttons[0];
        primary?.onPress?.();
      }
    } else {
      window.alert(`${title}\n\n${message}`);
    }
  }
  return;
}
// eslint-disable-next-line @typescript-eslint/no-require-imports
const { Alert } = require('react-native');
Alert.alert(title, message, buttons);
}

/** 仅显示消息的简化版本, 等价于 showAlert(title, message) */
export function showMessage(title: string, message: string): void {
  showAlert(title, message);
}

// ===== 文件: lib/utils/sse-parser.ts =====
/**
 * SSE 响应内容分段解析器
 * 将 Agent 回复中的不同内容段分离:
 * - 核心文本: 主要教学内容 + 核心要点 + 场景举例
 * - 白板内容: 🖍️ **白板图示** 后的代码块
 * - 引导思考: 💡 **引导思考** 用于触发讨论
 */

export interface ParsedContent {
  /** 核心教学内容 (包含要点和举例) */
}

```

侧伴移动端软件 V1.0.0 第 59 页

侧伴移动端软件 V1.0.0 第 60 页

```
return codeBlockMatch[0]
```

```

        .replace(/`\\w*\n?/, '')
        .replace(/`\\$/g, '')
        .trim();
    }

    /**
     * 从引导思考中提取讨论主题
     */
    export function extractDiscussionTopic(thinkingPrompt: string): string {
        if (!thinkingPrompt) return '讨论话题';
        // 提取引导思考中的核心问题 (去除emoji和标题)
        const lines = thinkingPrompt.split('\n').filter(l => l.trim());
        // 跳过标题行, 找问题内容
        for (let i = 1; i < lines.length; i++) {
            const line = lines[i].trim();
            if (line && !line.startsWith('💡') && !line.startsWith('😞') && !line.startsWith('***')) {
                return line.replace(/^[: :]\s*/, '');
            }
        }
        return lines[0]?.replace(/(?:💡|😞)\s*\*\*引导思考\[^\]*\*\*[[: :]\s*/, '') || '讨论话题';
    }
    // ===== 文件: assets/tailwind.config.js =====
    /**
     * EduDash Tailwind CSS Configuration
     * 提取自 packages/mobile/html/index.html
     */

    module.exports = {
        darkMode: "class",
        theme: {
            extend: {
                colors: {
                    "primary": "#ec5b13",
                    "background-light": "#f8f6f6",
                    "background-dark": "#221610",
                },
                fontFamily: {
                    "display": ["Public Sans", "sans-serif"]
                },
                borderRadius: {
                    "DEFAULT": "0.25rem",
                    "lg": "0.5rem",
                    "xl": "0.75rem",
                    "full": "9999px"
                },
            },
        },
        plugins: ["forms", "container-queries"],
    }

```